React.js 100+ Theory Interview Question and Answers

Introduction of react.js:

Q1. What is ReactJS?

Answer:

1. ReactJS is an open-source **JavaScript library** for building user interfaces (UIs).
2. It is commonly used for creating **single-page applications (SPAs)**.
3. Developed and maintained by **Facebook**.
4. Helps developers build **reusable UI components**.
5. Efficiently manages the **state** of applications.

Q2. What are the main features of ReactJS?

Answer:
The main features of ReactJS are:

1. **Component-Based Architecture**: Applications are built using small, reusable components.
2. **Virtual DOM**: React uses a virtual representation of the DOM to improve performance.
3. **Declarative UI**: Developers describe how the UI should look, and React takes care of rendering and updating.
4. **Unidirectional Data Flow**: Data flows in a single direction, making debugging and state management easier.
5. **JSX**: A syntax extension that allows writing HTML-like code in JavaScript.

Q3. How is React different from other JavaScript frameworks?

Answer:

- React is a **library**, not a full framework, which means it focuses on the **view layer** only.
- Unlike Angular or Vue, React relies on third-party libraries for routing and state management (like React Router and Redux).
- React's **virtual DOM** ensures better performance compared to direct DOM manipulation in some frameworks.

Q4. What are the advantages of using ReactJS?

Answer:

1. **Reusable Components**: React components can be reused across the application, saving development time.
2. **Performance**: The virtual DOM and efficient rendering improve performance.
3. **Strong Community**: React has a large developer community and ecosystem.
4. **Flexibility**: React can be used with other libraries or frameworks.
5. **SEO-Friendly**: React can render on the server side, making it SEO-friendly.

Virtual DOM:

Q5. What is the Virtual DOM in ReactJS?

Answer:
The **Virtual DOM** is a lightweight, in-memory representation of the **real DOM**. React uses the Virtual DOM to efficiently manage changes in the UI by:

1. Maintaining a virtual copy of the real DOM.
2. Comparing the virtual DOM with the real DOM to identify changes (a process called **diffing**).
3. Updating only the changed parts in the real DOM.

Q6. How does the Virtual DOM work in React?

Answer:

1. **Render**: React creates a Virtual DOM tree when a component is rendered.
2. **Diffing**: When the UI changes, React compares the new Virtual DOM tree with the previous one.
3. **Reconciliation**: React calculates the differences and updates only the affected parts of the real DOM.

```
const element = <h1>Hello, React!</h1>;
ReactDOM.render(element, document.getElementById("root"));
```

If element changes to <h1>Hello, World!</h1>, React updates only the text content, not the entire <h1> tag

Q7. How is the Virtual DOM different from the Real DOM?

Answer:

Aspect	Virtual DOM	Real DOM
Type	In-memory representation of the DOM	Actual browser DOM
Speed	Fast updates	Slower updates
Usage	Used by React for efficient rendering	Directly used by the browser to display UI
Manipulation	React handles it internally	Developers manipulate it directly

Q8. Is the Virtual DOM unique to ReactJS?

Answer:
No, the Virtual DOM concept is not unique to React. Other frameworks like Vue.js also use a

Virtual DOM for efficient UI updates. However, React popularized the concept and provides a highly optimized implementation.

JSX and Components:

Q9 . What is JSX in ReactJS?

Answer:
JSX (JavaScript XML) is a syntax extension for JavaScript that allows you to write HTML-like code directly within JavaScript. For example:

```
const element = <h1>Hello, React!</h1>;
```

This JSX code is converted into JavaScript code using tools like Babel. It makes the code more readable and easy to maintain.

Q10. How does JSX work under the hood?

Answer:
JSX is not executed directly by the browser. Instead, it is **transpiled** into JavaScript using tools like Babel. For example:

```
const element = <h1>Hello, JSX!</h1>;
```

Transpiles to:

```
const element = React.createElement("h1", null, "Hello, JSX!");
```

Q11. What are some rules for using JSX?

Answer:
Rules for JSX include:

1. Must return a **single parent element**; wrap siblings in or <React.Fragment>.
2. Use className instead of class for CSS classes.
3. Close all tags, including self-closing ones (e.g., <img />).
4. JavaScript expressions are wrapped in {}.
5. Avoid conditionals and loops directly in JSX; use helper methods instead.

Q12. Can you use JavaScript expressions in JSX?

Answer:
Yes, JSX allows JavaScript expressions:

1. Wrap expressions inside {} to embed them in JSX.
2. Supports operations like calculations ({a + b}).
3. Allows function calls ({formatDate(date)}).
4. Conditional rendering using ternary ({isLoggedIn ? 'Welcome' : 'Login'}).
5. Works seamlessly with arrays ({items.map((item) => <li>{item}</li>)}).

React.js 100+ Theory Interview Question and Answers

Q13. What are Components in ReactJS?

Answer:
Components in React:

1. Are **reusable UI building blocks** for applications.
2. Can be functional or class-based.
3. Define how a specific part of the UI should look and behave.
4. Manage their own **state and lifecycle**.
5. Help organize large applications into smaller, maintainable pieces.

Q14. What are the types of React components?

Answer:
Types of React components:

1. **Functional Components**: Simple JavaScript functions that return JSX.
2. **Class Components**: JavaScript classes extending React.Component.
3. **Higher-Order Components (HOCs)**: Functions that enhance a component.
4. **Presentational Components**: Focus only on rendering the UI.
5. **Container Components**: Handle logic and state management.

Q15. What is the difference between Functional and Class Components?

Answer:

Feature	Functional Components	Class Components
Definition	Functions that return JSX	Classes extending React.Component
State	Use Hooks (e.g., useState)	Use this.state
Lifecycle	Managed with Hooks	Explicit lifecycle methods (e.g., componentDidMount)
Simplicity	Simpler and more concise	More verbose
Performance	Slightly better due to optimizations	Slightly heavier due to inheritance

Q16. How do components communicate in ReactJS?

Answer:
Communication in React happens via:

1. **Parent to Child**: Using props to pass data.
2. **Child to Parent**: Passing a callback function as a prop.
3. **Sibling Communication**: Using a shared parent component to manage state.
4. **Global State Management**: Using context or libraries like Redux.

5. **Event Handlers**: Passing events from one component to another.

Q17. What is the role of props in components?

Answer:
props are:

1. Short for **properties** passed from a parent to a child component.
2. Used to make components **dynamic** and reusable.
3. Immutable in the child; changes must happen in the parent.
4. Accessed in functional components as props and in class components as this.props.
5. Useful for passing **data**, **functions**, and **event handlers** to child components.

Q18. What are controlled and uncontrolled components?

Answer:

Feature	Controlled Component	Uncontrolled Component
Data Handling	Controlled by React state	Controlled by the DOM
Example	<input value={stateValue} />	<input defaultValue="value" />
Control Level	React handles all updates	Browser handles updates
Use Case	Best for complex forms and validation	Best for simple inputs
State Management	Requires state and handlers	Minimal to no React involvement

Q19. What are the advantages of using components in ReactJS?

Answer:

1. **Reusability**: Build once, reuse anywhere in the app.
2. **Readability**: Makes the application modular and organized.
3. **Testability**: Individual components are easier to test.
4. **Maintainability**: Easier to debug and update smaller pieces of the app.
5. **Scalability**: Helps manage large-scale applications by dividing them into smaller units.

Props:

Q20. What are the main characteristics of props?

Answer:

1. **Immutable**: Cannot be modified by the child component.
2. **Unidirectional**: Data flows from parent to child.
3. **Dynamic**: Make components adaptable to different data inputs.
4. **Read-Only**: A child component can read but not change props.
5. **Mandatory or Optional**: Can be defined as required or optional using PropTypes.

Q21. What is the difference between props and state?

Answer:

Aspect	Props	State
Definition	Passed from parent to child	Managed within the component
Mutability	Immutable	Mutable
Responsibility	Controlled by parent	Controlled by the component itself
Purpose	Used for passing data	Used for managing dynamic data
Lifecycle	Available throughout the component's lifecycle	Can be updated over time

Q22. What is children in props?

Answer:
The children prop:

1. Refers to the **content** passed between a component's opening and closing tags.
2. Enables nesting of components.

```
function Container({ children }) {
  return <div className="container">{children}</div>;
}

<Container>
  <p>This is child content!</p>
</Container>
```

React automatically assigns the nested content to props.children.
Useful for creating **wrapper** or **layout** components.

Q23. What are the advantages of using props in React?

Answer:

1. **Reusable Components**: Enables components to work with different data inputs.
2. **Dynamic UIs**: Makes components flexible and responsive to changing data.
3. **Unidirectional Data Flow**: Ensures predictable and easier-to-debug component behavior.
4. **Integration**: Pass data, event handlers, or functions seamlessly between components.
5. **Type Checking**: PropTypes helps enforce correct data types, reducing bugs.

Lifecycle Methods:

Q24. What are React lifecycle methods?

Answer:
Lifecycle methods are special methods in React that allow you to:

1. Control what happens at different stages of a component's lifecycle.
2. Manage actions like data fetching, DOM updates, and cleanups.
3. React to changes in props or state.
4. Lifecycle is divided into three phases:
   - **Mounting**: When the component is first added to the DOM.
   - **Updating**: When the component is re-rendered due to changes in props or state.
   - **Unmounting**: When the component is removed from the DOM.
5. Functional components handle lifecycle methods using **React Hooks**.

Q25. What are the lifecycle phases in React?

Answer:
The lifecycle phases are:

1. **Mounting Phase**:
   - Occurs when the component is created and inserted into the DOM.
   - Methods: constructor(), getDerivedStateFromProps(), render(), componentDidMount().
2. **Updating Phase**:
   - Happens when props or state change, causing the component to re-render.
   - Methods: getDerivedStateFromProps(), shouldComponentUpdate(), render(), getSnapshotBeforeUpdate(), componentDidUpdate().
3. **Unmounting Phase**:
   - Occurs when the component is removed from the DOM.
   - Methods: componentWillUnmount().
4. **Error Handling Phase**:
   - Handles errors in child components.
   - Methods: componentDidCatch() and getDerivedStateFromError().

Q26. How do you handle errors in React lifecycle?

Answer:
React provides the **Error Handling phase**:

1. **componentDidCatch(error, info)**:

- Catches errors in child components and logs them.

```
componentDidCatch(error, info) {
  console.error("Error occurred:", error, info);
}
```

2. getDerivedStateFromError(error):

- Updates state when an error occurs, enabling fallback UI.

Q27. What are React Hooks for lifecycle management?

Answer:
Functional components manage lifecycle using **Hooks**:

useEffect():

- Replaces componentDidMount, componentDidUpdate, and componentWillUnmount.
- Example:

```
useEffect(() => {
  console.log("Component mounted or updated");
  return () => console.log("Component unmounted");
}, [dependency]);
```

useState():

- Manages state in functional components, replacing this.state and setState.

useRef():

- Manages references, replacing getSnapshotBeforeUpdate.

Q28. What is the difference between componentDidMount and componentDidUpdate?

Answer:

Feature	componentDidMount	componentDidUpdate
When it Runs	After the component is mounted	After the component updates
Purpose	Initialize data, fetch API, add listeners	Update data or DOM based on new props/state
Runs On Render	Only runs once	Runs after every update

Feature	componentDidMount	componentDidUpdate
Typical Use Cases	Fetching data, starting animations	Updating data based on state changes

Q29. What are the limitations of lifecycle methods?

Answer:

1. **Complexity**: Managing lifecycle methods in class components can become hard to read.
2. **Verbose**: Requires multiple methods for similar actions (e.g., componentDidMount and componentDidUpdate).
3. **Hooks Limitation**: Functional components with Hooks might behave differently if not implemented carefully.
4. **Error Handling**: Limited to errors in child components, not asynchronous tasks.
5. **Deprecation**: Older lifecycle methods like componentWillReceiveProps are deprecated in favor of newer ones.

Q30. What are React Hooks?

Answer:
React Hooks are:

1. Special functions that allow you to use **state** and other React features in functional components.
2. Introduced in React 16.8 to eliminate the need for class components.
3. Simplify code by managing state and lifecycle in functional components.
4. Enable reuse of stateful logic between components.
5. Include built-in hooks like useState, useEffect, and useContext, and allow for creating custom hooks.

Q31. What are the rules of Hooks?

Answer:
React Hooks follow two main rules:

1. **Only Call Hooks at the Top Level**:
   - Do not use hooks inside loops, conditions, or nested functions.
2. **Only Call Hooks in React Functions**:
   - Use hooks only in functional components or custom hooks.

These rules ensure consistent execution and avoid unexpected behavior.

Q32. What are some commonly used built-in Hooks?

Answer:

1. **useState**: Manages state in functional components.

2. **useEffect**: Handles side effects like data fetching and subscriptions.
3. **useContext**: Accesses the value of a context.
4. **useRef**: Creates a reference to a DOM element or a mutable value.
5. **useReducer**: Manages complex state logic, similar to Redux.
6. **useMemo**: Optimizes performance by memoizing values.
7. **useCallback**: Memoizes functions to prevent unnecessary re-creations.
8. **useLayoutEffect**: Similar to useEffect, but fires synchronously after DOM mutations.

Q33. What are the advantages of using Hooks?

Answer:
Hooks provide the following benefits:

1. **Simplified Code**: Replace class-based lifecycle methods with cleaner functional logic.
2. **Reusable Logic**: Extract and reuse stateful logic via custom hooks.
3. **Better Readability**: Functional components with hooks are easier to understand.
4. **Improved Performance**: Avoid unnecessary re-renders with useMemo and useCallback.
5. **Backward Compatibility**: Works seamlessly alongside class components.

Q34. What is useState() in React?

Answer:
useState() is a React Hook that:

1. Allows functional components to manage state.
2. Returns two values:
    - The current state value.
    - A function to update the state.
3. Triggers a re-render when the state is updated.
4. Accepts an initial state as an argument.
5. Can handle any data type: string, number, array, or object.

```
const [count, setCount] = useState(0);
```

Q35. How does React handle asynchronous updates in useState()?

Answer:

1. useState() updates are **asynchronous**:
    - Multiple updates may be batched together for performance.
2. Access the **latest state** using the functional form

```
setState((prevState) => prevState + 1);
```

3. Directly accessing the state immediately after an update may result in outdated values.

Q36. What are the common pitfalls of useState()?

Answer:

1. **Direct Updates**: Avoid directly modifying state objects or arrays.
   - Use immutable updates instead.
2. **Asynchronous Updates**: State may not update immediately; rely on functional updates for dependent changes.
3. **Too Many States**: Splitting state into too many useState() calls can lead to complexity.
4. **Re-Renders**: Frequent state updates can cause unnecessary re-renders.
5. **State Reset**: If not handled carefully, state can reset during parent re-renders.

Q37. Can you use multiple useState() calls in a single component?

Answer:
Yes, multiple useState() calls are allowed:

1. Each useState() call manages a separate piece of state.

```
const [count, setCount] = useState(0);
const [name, setName] = useState("React");
```

2. This keeps state modular and easier to manage.

useEffect:

Q38. What is the syntax of **useEffect**?

Answer:
The syntax of useEffect is:

```
useEffect(() => {
  // Effect logic
  return () => {
    // Cleanup logic (optional)
  };
}, [dependencies]);
```

The **callback** runs after rendering.
The **dependency array** controls when the effect runs.
The **cleanup function** runs during unmounting or before re-running the effect.

Q39. What are side effects in React?

Answer:
Side effects in React are operations that:

1. Interact with the outside world, like fetching data from an API.
2. Perform **manual DOM updates** or animations.
3. Add or remove **event listeners**.
4. Manage **subscriptions** or timers.
5. Update state or execute functions that affect other parts of the app.

Q40. How does the dependency array in useEffect work?

Answer:
The dependency array:

1. Specifies variables that useEffect should watch for changes.
2. **Empty Array ([])**: Effect runs only once after the initial render.
3. **Array with Dependencies**: Effect re-runs when any dependency changes. Example:

```
useEffect(() => {
  console.log(count);
}, [count]);
```

4. **No Array**: Effect runs after every render (default behavior).
5. Reduces unnecessary re-renders by limiting effect execution.

Q41. What is the cleanup function in useEffect?

Answer:
The cleanup function:

1. Is optional and runs before the effect is re-executed or when the component unmounts.
2. Used to:
   - Remove event listeners.
   - Clear intervals or timeouts.
   - Cancel subscriptions or API requests.

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log("Timer running");
```

```
  }, 1000);

  return () => {
    clearInterval(timer); // Cleanup
  };
}, []);
```

Q42. How is useEffect used for data fetching?

Answer:
To fetch data using useEffect:

1. Call the API inside the effect callback.
2. Use an empty dependency array to fetch data only once.
3. Handle errors and loading states as needed.

```
useEffect(() => {
  async function fetchData() {
    const response = await fetch("https://api.example.com/data");
    const data = await response.json();
    console.log(data);
  }
  fetchData();
}, []);
```

Q43. What are the common pitfalls of using useEffect?

Answer:

1. **Missing Dependencies**:
   - Omitting dependencies can cause stale values in the effect.
2. **Infinite Loops**:
   - Updating state inside the effect without controlling dependencies can lead to infinite re-renders.
3. **Overuse**:
   - Avoid putting too much logic into a single effect; split into multiple effects if needed.
4. **Memory Leaks**:
   - Forgetting cleanup can lead to resource leaks.
5. **Redundant Calls**:
   - Ensure the effect runs only when necessary by properly configuring the dependency array.

useRef:

Q44. What is useRef in React?

Answer:
useRef is a React Hook that:

1. Creates a mutable object that persists across renders.
2. Returns an object with a current property that can hold a value or a DOM element reference.
3. Does not trigger re-renders when the value of current changes.
4. Often used to interact with DOM elements directly.
5. Useful for storing values that do not need to cause re-renders, like timers or previous states.

Q45. How is useRef used to reference DOM elements?

Answer:
useRef is used to access a DOM element directly:

Create a ref using useRef

```
const inputRef = useRef(null);
```

Attach the ref to a DOM element using the ref attribute

```
<input ref={inputRef} type="text" />;
```

Access the DOM element

```
inputRef.current.focus();
```

Q46. What are the key differences between useRef and useState?

Answer:

Feature	useRef	useState
Triggers Re-render	No	Yes
Primary Use	Storing mutable values, referencing DOM	Managing component state
Value Type	Mutable, accessed via .current	Immutable, updated using setter function
Persistence	Persists across renders	Re-initializes on re-render
Performance	Lightweight, no re-render overhead	May cause performance hits with frequent updates

Q47. How does useRef help track previous state or props?

Answer:
useRef can be used to store the previous value of state or props:

1. Example:

```
function Component({ value }) {
  const prevValue = useRef();

  useEffect(() => {
    prevValue.current = value;
  });

  return <div>Previous Value: {prevValue.current}</div>;
}
```

The useRef value updates on every render but does not trigger re-renders.

Q48. How does useRef improve performance in React?

Answer:
useRef improves performance by:

1. Avoiding re-renders when its value changes.
2. Acting as a **lightweight storage** for data that does not impact UI updates.
3. Preventing re-initialization of values on every render.
4. Allowing direct manipulation of DOM elements without triggering state changes.
5. Reducing the computational overhead by persisting mutable values across renders.

Q49. What are the limitations of useRef?

Answer:

1. **No Re-render Trigger**:
   - Changing current does not cause a component to re-render.
2. **Not Reactive**:
   - Values stored in useRef are not tracked by React's rendering system.
3. **Improper State Replacement**:
   - Should not be used to manage UI-driven state that requires rendering updates.
4. **Limited Usage**:
   - Primarily used for DOM manipulation or non-rendered values.
5. **Can Lead to Bugs**:
   - Misuse of useRef for reactive state can cause unexpected behavior.

useReducer:

Q50. What is useReducer in React?

Answer:
useReducer is a React Hook that:

1. Manages complex state logic by using a reducer function.
2. Returns the current state and a dispatch function to update the state.

3. Works similarly to useState, but is better suited for managing **complex or multiple state transitions**.
4. Requires a reducer function that determines how the state should change based on an action.
5. Is inspired by the Redux pattern but operates at the component level.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Q51. What is the syntax of useReducer?

Answer:
The syntax of useReducer is:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

reducer: A function that takes the current state and an action, and returns the new state.
initialState: The initial value of the state.
state: The current state value.
dispatch: A function to send actions to the reducer.

Q52. How is useReducer different from useState?

Answer:

Aspect	useReducer	useState
Use Case	Complex state with multiple transitions	Simple state logic
State Update	Centralized in a reducer function	Updates with a direct setter function
Readability	Easier for managing multiple state actions	Better for simple state updates
Scalability	Handles large and complex state efficiently	May become cumbersome for complex logic
Syntax	Requires a reducer and dispatch	Simpler to use

Q53. How do you handle multiple actions with useReducer?

Answer:
Define multiple cases in the reducer function:

```
function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { ...state, count: state.count + 1 };
    case "decrement":
      return { ...state, count: state.count - 1 };
    case "reset":
      return { ...state, count: 0 };
    default:
      return state;
  }
}
```

Use dispatch to send actions:

```
dispatch({ type: "increment" });
dispatch({ type: "reset" });
```

Q54. Can you combine useReducer with useContext?

Answer:
Yes, combining useReducer with useContext is common for global state management:

1. Create a context:

```
const CounterContext = React.createContext();
```

2. Use useReducer to manage the state:

```
const initialState = { count: 0 };
const [state, dispatch] = useReducer(reducer, initialState);
```

3. Provide the state and dispatch:

```
<CounterContext.Provider value={{ state, dispatch }}>
  {children}
</CounterContext.Provider>;
```

4. Consume the context in child components using useContext:

```
const { state, dispatch } = useContext(CounterContext);
```

Q55. How do you optimize performance with useReducer?

Answer:

1. **Memoize Dispatch Functions**: Use useCallback to avoid unnecessary re-renders.

2. **Split Reducers**: Divide a large reducer into smaller reducers for better maintainability.
3. **Minimize State Updates**: Only update state when necessary.
4. **Lazy Initialization**: Use a function to compute the initial state

```
const [state, dispatch] = useReducer(reducer, undefined, initializeState);
```

5. **Combine with Context**: Use useContext for global state to reduce prop drilling.

useMemo:

Q56. What is useMemo in React?

Answer:
useMemo is a React Hook that:

1. Memoizes a **value** to avoid re-computation on every render.
2. Accepts a computation function and a dependency array.
3. Returns the memoized result unless dependencies change.
4. Optimizes performance for expensive calculations.
5. Often used with useCallback to memoize functions or values.

Example:

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

Q57. How does useMemo optimize performance?

Answer:
useMemo optimizes performance by:

1. Caching the result of expensive computations.
2. Preventing unnecessary calculations on every render.
3. Only recalculating when dependencies change.
4. Reducing computational overhead for intensive tasks.
5. Useful for performance-critical components with complex logic.

Q58. What is the syntax of useMemo?

Answer:
The syntax of useMemo is:

```
const memoizedValue = useMemo(() => computationFunction, [dependencies]);
```

computationFunction: The function whose result should be memoized.
dependencies: An array of values that determine when the computation should re-run.

useCallback:

Q59. What is useCallback in React?

Answer:
useCallback is a React Hook that:

1. Memoizes a **function** so that it retains the same reference between renders.
2. Prevents re-creation of the function unless its dependencies change.
3. Improves performance by reducing unnecessary re-renders of child components.
4. Works best when used with React.memo to optimize functional components.
5. Helps avoid redundant computations in performance-critical applications

Q60. What is the syntax of useCallback?

Answer:
The syntax of useCallback is

```
const memoizedCallback = useCallback(callbackFunction, [dependencies]);
```

callbackFunction: The function to be memoized.
dependencies: An array of values that the function depends on. When any value changes, the function is re-created.

Q61. How does useCallback help with React.memo?

Answer:

1. React.memo prevents re-rendering of a child component if its props haven't changed.
2. If a parent re-creates a function passed as a prop, React.memo detects it as a change and re-renders the child.
3. Using useCallback, the function retains its reference, ensuring the child doesn't re-render unnecessarily.
4. Example:

```
const Child = React.memo(({ onClick }) => {
  console.log("Child rendered");
  return <button onClick={onClick}>Click Me</button>;
});

const Parent = () => {
  const handleClick = useCallback(() => {
    console.log("Button clicked");
  }, []);
  return <Child onClick={handleClick} />;
};
```

Q62. What is the difference between useCallback and useMemo?

Answer:

Aspect	useCallback	useMemo
Purpose	Memoizes a **function**	Memoizes a **value**
Return Value	A memoized function	A memoized value
Use Case	Avoiding function re-creation	Avoiding value re-computation
Dependencies	Tracks changes in function dependencies	Tracks changes in value dependencies

useContext:

Q63. What is useContext in React?

Answer:
useContext is a React Hook that:

1. Allows components to access the value of a React **Context** without needing a Consumer.
2. Simplifies sharing **global data** like themes, authentication, or settings across components.
3. Accepts a **context object** created by React.createContext.
4. Eliminates **prop drilling** by providing data directly to deeply nested components.
5. Works seamlessly with the Context API for efficient state management.

```
const ThemeContext = React.createContext("light");
const theme = useContext(ThemeContext);
```

Q64. What is the syntax of useContext?

Answer:
The syntax of useContext is:

```
const value = useContext(ContextObject);
```

ContextObject: The context object returned by React.createContext.
value: The value provided by the nearest Context.Provider.

Q65. How do you create and use a Context with useContext?

Answer:

1. **Create Context**

```
const UserContext = React.createContext();
```

2. **Provide Context**:

```
<UserContext.Provider value={{ name: "Alice", age: 25 }}>
  <App />
</UserContext.Provider>;
```

3. Consume Context

```
const user = useContext(UserContext);
console.log(user.name); // "Alice"
```

Q66. Can useContext be combined with useReducer?

Answer:
Yes, combining useContext and useReducer is common for managing global state:

1. Use useReducer to handle complex state logic.
2. Provide the state and dispatch function via Context.

```
const AppContext = React.createContext();

const AppProvider = ({ children }) => {
  const [state, dispatch] = useReducer(reducer, initialState);
  return (
    <AppContext.Provider value={{ state, dispatch }}>
      {children}
    </AppContext.Provider>
  );
};
```

3. Consume both state and dispatch in any component

```
const { state, dispatch } = useContext(AppContext);
```

Q67. What is the difference between useContext and prop drilling?

Answer:

Aspect	useContext	Prop Drilling
Purpose	Access global data without passing props	Passes data explicitly through each level
Ease of Use	Simplifies code	Becomes complex with deeply nested trees
Flexibility	Easily share data across components	Requires changes at every intermediate level
Reusability	Provides reusable global state	Data is tied to specific prop chains

Q68. What is React Router DOM?

Answer:
React Router DOM is:

1. A library for **routing** in React applications.
2. Enables navigation between different pages or components without reloading the browser.
3. Provides components like <BrowserRouter>, <Routes>, and <Route> to define and manage routes.
4. Allows for **dynamic routing**, meaning routes can change based on application state.
5. Supports both **client-side rendering (CSR)** and **server-side rendering (SSR)**.

Q69. What are the core components of React Router DOM?

Answer:

1. **<BrowserRouter>**:
   - Wraps the entire application to enable routing.
2. **<Routes>**:
   - Replaces <Switch> in React Router v6 and manages route matching.
3. **<Route>**:
   - Defines individual routes and the components to render.
4. **<Link>**:
   - Provides navigation between routes without reloading the page.
5. **<Navigate>**:
   - Redirects users to a different route programmatically.

Q70. How do you define routes in React Router DOM?

Answer:
Routes are defined using <Routes> and <Route>:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
```

path: Specifies the URL path for the route.
element: Specifies the component to render.

Q71. What is the difference between <Link> and <a> in React Router?

Answer:

Aspect	<Link>	<a>
Usage	Used for navigation in React Router	Standard HTML link
Page Reload	No reload (client-side navigation)	Reloads the entire page
Performance	Faster as it avoids full page reload	Slower due to full page reload
Routing	Works seamlessly with React Router	Does not interact with React Router

Q72. How does nested routing work in React Router DOM?

Answer:

1. **Nested Routes** allow child routes to be rendered inside parent routes.
2. Defined using a parent <Route> with child <Route> elements.

Example:

```
<Routes>
  <Route path="/dashboard" element={<Dashboard />}>
    <Route path="profile" element={<Profile />} />
    <Route path="settings" element={<Settings />} />
  </Route>
</Routes>;
```

3.The child routes inherit the parent route's path (e.g., /dashboard/profile).

Q73. What are useNavigate and useLocation hooks in React Router?

Answer:

1. **useNavigate**:
   - Allows programmatic navigation to a different route.

Example:

```
const navigate = useNavigate();
navigate("/about");
```

2. useLocation:

- Provides information about the current route.

- Example:

```
const location = useLocation();
console.log(location.pathname); // Current path
```

Q74. How does React Router handle route parameters?

Answer:

1. Route parameters are placeholders in the route path (e.g., /user/:id).
2. Access parameters using useParams hook:

```
<Route path="/user/:id" element={<User />} />;
```

3. Inside the User component:

```
import { useParams } from "react-router-dom";
const { id } = useParams();
console.log(id); // Captures the dynamic parameter
```

Q75. What are the best practices for using React Router DOM?

Answer:

- Use **nested routes** to simplify and organize complex route hierarchies.
- Always include a **404 page** using a wildcard route:
  ```
  <Route path="*" element={<NotFound />} />;
  ```
- Memoize route components with React.memo for better performance.
- Use **lazy loading** for routes to improve initial load time.
- Use useNavigate for programmatic navigation instead of manipulating the URL directly.

Q76. What is Axios?

Answer:
Axios is:

1. A promise-based HTTP client for JavaScript.
2. Used to make **HTTP requests** (GET, POST, PUT, DELETE) to APIs in web applications.
3. Supports features like **interceptors**, **timeout**, and automatic **JSON data transformation**.
4. Works both in the **browser** and **Node.js** environments.
5. Easier to use compared to the native fetch API, with enhanced capabilities

Q77. Why is Axios preferred over the fetch API?

Answer:
Axios is often preferred over fetch because:

1. **Simpler Syntax**: Automatically parses JSON responses.

2. **Error Handling**: Handles HTTP errors more cleanly, unlike fetch which resolves even on errors.
3. **Timeout Support**: Allows setting timeouts for requests.
4. **Interceptors**: Lets you modify requests or responses globally.
5. **Cancelation**: Supports request cancelation with CancelToken.

Q78. What are Axios interceptors?

Answer:
Interceptors allow you to modify requests or responses globally:

Request Interceptor:

- Modify requests before they are sent.

```
axios.interceptors.request.use((config) => {
  config.headers.Authorization = "Bearer token";
  return config;
});
```

Response Interceptor:

- Handle or transform responses globally.

```
axios.interceptors.response.use(
  (response) => response,
  (error) => {
    console.error("Global error handling:", error);
    return Promise.reject(error);
  }
);
```

Q79. How do you set a base URL in Axios?

Answer:
Set a base URL for all requests:

```
const api = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com",
});

api.get("/posts").then((response) => console.log(response.data));
```

Use axios.create() to configure a custom Axios instance.
Access endpoints by appending paths to the base URL.

Q80. How do you handle timeouts with Axios?

Answer:
Set a timeout for requests:

```
axios
  .get("https://jsonplaceholder.typicode.com/posts", {
    timeout: 5000, // Timeout in milliseconds
  })
  .then((response) => console.log(response.data))
  .catch((error) => console.error("Request timed out:", error));
```

The timeout property specifies the maximum time for a request.
Requests exceeding the timeout period are automatically canceled.

Q81. How do you cancel a request in Axios?

Answer:
Cancel a request using CancelToken:

```
import axios from "axios";

const source = axios.CancelToken.source();

axios
  .get("https://jsonplaceholder.typicode.com/posts", {
    cancelToken: source.token,
  })
  .then((response) => console.log(response.data))
  .catch((error) => {
    if (axios.isCancel(error)) {
      console.log("Request canceled:", error.message);
    } else {
      console.error("Error:", error);
    }
  });

// Cancel the request
source.cancel("Request canceled by the user.");
```

Q82. What are some best practices when using Axios?

Answer:

1. **Centralize API Calls**:
   - Create an Axios instance to manage base configurations.
2. **Handle Errors Globally**:
   - Use response interceptors for consistent error handling.

3. **Use async/await**:
   - Simplifies asynchronous code.

```
const fetchData = async () => {
  try {
    const response = await axios.get("/posts");
    console.log(response.data);
  } catch (error) {
    console.error("Error fetching data:", error);
  }
};
```

4. **Optimize Performance**:
   - Use caching or debounce requests to reduce server load.

5. **Secure Requests**:
   - Add authentication headers using request interceptors.

Lazy Loading:

Q83. What is lazy loading in React?

Answer:
Lazy loading is a technique in React that:

1. Delays the loading of components or resources until they are needed.
2. Improves the **initial loading time** of an application by splitting the code into smaller chunks.
3. Uses React.lazy() and Suspense for dynamic imports of components.
4. Is especially useful for large components, pages, or modules that are not always visible.
5. Enhances performance by loading critical resources first and deferring non-essential ones.

Q84. What are the benefits of lazy loading?

Answer:

1. **Performance Optimization**: Reduces the initial load time by deferring non-critical resources.
2. **Smaller Bundle Size**: Splits the application into smaller chunks for efficient loading.
3. **Enhanced User Experience**: Loads visible components first, improving perceived performance.
4. **Dynamic Imports**: Allows loading only the parts of the application required for a specific route or feature.

5. **Resource Management**: Reduces server and network load by fetching resources on demand.

Q85. What is the syntax for lazy loading a component?

Answer:
The syntax for lazy loading is:

```
const LazyComponent = React.lazy(() => import("./LazyComponent"));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <LazyComponent />
    </Suspense>
  );
}
```

- Use React.lazy() for dynamic imports.
- Wrap the component with Suspense and provide a fallback UI.

Q86. How does lazy loading work with React Router?

Answer:
Lazy loading is used with React Router to load route-specific components:

1. Dynamically import components:

```
const About = React.lazy(() => import("./About"));
const Contact = React.lazy(() => import("./Contact"));
```

2. Define routes with lazy-loaded components:

```
<BrowserRouter>
  <Routes>
    <Route
      path="/about"
      element={
        <Suspense fallback={<div>Loading...</div>}>
          <About />
        </Suspense>
      }
    />
    <Route
      path="/contact"
      element={
        <Suspense fallback={<div>Loading...</div>}>
          <Contact />
        </Suspense>
      }
```

```
    />
  </Routes>
</BrowserRouter>;
```

Q87. How do you handle errors in lazy-loaded components?

Answer:
Handle errors using an **Error Boundary**:

1. Create an error boundary component:

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  render() {
    if (this.state.hasError) {
      return <div>Something went wrong!</div>;
    }
    return this.props.children;
  }
}
```

2. Wrap the lazy-loaded component

```
<ErrorBoundary>
  <Suspense fallback={<div>Loading...</div>}>
    <LazyComponent />
  </Suspense>
</ErrorBoundary>;
```

Q88. How is lazy loading different from code splitting?

Answer:

Aspect	Lazy Loading	Code Splitting
Definition	Defers loading until needed	Splits application code into smaller chunks
Purpose	Optimizes load time	Improves bundle management
Trigger	Occurs during runtime	Can occur during build or runtime
Dependency	Uses React.lazy and Suspense	Uses dynamic imports or bundlers

Q89. What are some best practices for lazy loading in React?

Answer:

1. **Use Meaningful Fallbacks**: Provide a clear loading indicator or skeleton UI for better UX.
2. **Combine with Code Splitting**: Use tools like Webpack to split code into optimized chunks.
3. **Lazy Load Routes**: Load route-specific components dynamically with React Router.
4. **Error Boundaries**: Handle errors gracefully using error boundaries.
5. **Monitor Performance**: Regularly check network and performance metrics to ensure lazy loading benefits.

Webpack in React:

Q90. Why is Webpack important in React applications?

Answer:
Webpack is important because:

1. It allows React applications to use **modern JavaScript** features by transpiling code.
2. Bundles multiple files into fewer HTTP requests, improving performance.
3. Handles **dependencies** between modules automatically.
4. Enables **dynamic imports** for lazy loading components.
5. Provides **hot module replacement (HMR)** for faster development.

Q91. **How do you configure Webpack in a React project?**

Answer:

1. Install Webpack and dependencies:

```
npm install webpack webpack-cli webpack-dev-server babel-loader @babel/core
@babel/preset-react html-webpack-plugin
```

2. Create a webpack.config.js file:

```
const path = require("path");
const HtmlWebpackPlugin = require("html-webpack-plugin");

module.exports = {
  entry: "./src/index.js",
  output: {
    path: path.resolve(__dirname, "dist"),
    filename: "bundle.js",
```

```
  },
  module: {
    rules: [
      {
        test: /\.js$/,
        exclude: /node_modules/,
        use: "babel-loader",
      },
      {
        test: /\.css$/,
        use: ["style-loader", "css-loader"],
      },
    ],
  },
  plugins: [new HtmlWebpackPlugin({ template: "./src/index.html" })],
  mode: "development",
};
```

3. Run the development server

```
npx webpack serve
```

NPM vs NPX:

Q92. How does NPM differ from NPX?

Answer:

Aspect	NPM	NPX
Purpose	Installs packages	Executes packages
Installation	Requires manual installation of tools	Automatically fetches and executes tools
Usage	Manages project dependencies	Runs one-time commands or utilities
Global Packages	Often installs globally	Avoids global installations
Command Examples	npm install create-react-app -g	npx create-react-app my-app

Q93. Why is NPX useful?

Answer:
NPX is useful because:

1. **No Global Installation Needed**:
   - Runs packages without globally installing them, saving disk space.
2. **Always Up-to-Date**:
   - Ensures the latest version of a tool is used.
3. **Ease of Use**:
   - Simplifies one-off tasks like scaffolding projects or running scripts.
4. **Reduces Version Conflicts**:
   - Fetches the exact version required for the task.
5. **Temporary Usage**:
   - Removes the package after execution.

Q94. When should you use NPM instead of NPX?

Answer:
Use NPM when:

1. Installing **dependencies** for long-term use in a project.

```
npm install react
```

2. Managing project configurations via package.json.
3. Installing packages globally for repeated use.
4. Updating installed packages.
5. Running project-specific scripts via npm run.

Q96. When should you use NPX instead of NPM?

Answer:
Use NPX when:

1. Running a tool or command temporarily.

```
npx create-react-app my-app
```

2. Avoiding **global installations** for one-off tasks.
3. Testing or running specific package versions.
4. Quickly bootstrapping a new project.
5. Ensuring the latest version of a CLI tool is used.

Q97. What are the limitations of NPX?

Answer:

1. **Temporary Installation**:
   - Packages are not retained for reuse, requiring a fresh download each time.
2. **Limited for Long-Term Dependencies**:
   - Not suitable for managing project dependencies.
3. **Requires Internet**:
   - Needs an active internet connection to fetch packages.
4. **Version-Specific Issues**:

- May accidentally fetch an unintended version if not specified explicitly.

Q98. What is the difference between state and props in React?

Answer:

Aspect	State	Props
Definition	Data managed within a component	Data passed from parent to child
Mutability	Mutable (can be updated)	Immutable (read-only)
Scope	Local to the component	Available to child components
Purpose	For dynamic, interactive behavior	For passing data and configuration
Access	Managed via useState or this.state	Accessed via props in child components

Q99. Explain the concept of "declarative programming" in React ?

Answer:

Declarative programming in React means describing *what* the UI should look like based on the current state, rather than specifying *how* to build it step by step. React takes care of efficiently updating and rendering the UI when the state or data changes.

Key Points:

1. **Focus on Outcome**:
   - You describe the desired UI (e.g., <button>{text}</button>), and React figures out how to create or update the DOM.
2. **State-Driven**:
   - React components re-render automatically when the underlying state or props change.
3. **Cleaner Code**:
   - Declarative programming removes the need for direct DOM manipulation (e.g., document.getElementById).
4. **Comparison to Imperative**:
   - Imperative programming specifies *how* to perform each step to achieve a goal, while declarative specifies *what* the end result should be.

Example:

Declarative:

```
function App() {
  const [count, setCount] = useState(0);

  return <button onClick={() => setCount(count + 1)}>Count: {count}</button>;
}
```

Imperative (manipulating DOM manually):

```
const button = document.createElement("button");
button.addEventListener("click", () => {
  count += 1;
  button.innerHTML = `Count: ${count}`;
});
document.body.appendChild(button);
```

Q100. What is the difference between stateful and stateless components?

Answer:

Aspect	Stateful Component	Stateless Component
Definition	Manages its own local state	Does not manage any state internally
Reactivity	Reacts to state changes by re-rendering	Relies on props for reactivity
Use Case	For dynamic, interactive components	For static or reusable presentational components
Syntax	Requires useState or this.state	Simple functional components
Example	Counter component with useState	Button component that receives onClick as a prop

Q101. What is the difference between <Routes> and <Switch> in React Router?

Answer:

Aspect	<Routes> (React Router v6)	<Switch> (React Router v5 and earlier)
Version	Introduced in React Router v6	Used in React Router v5 and earlier
Behavior	Matches the **most specific route**	Matches the **first route** that fits
Support for element	Requires element for route rendering	Uses component or render props
Nested Routes	Fully supports nesting with relative paths	Limited nesting capabilities
Syntax Update	<Route path="/" element={<Home />} />	<Route path="/" component={Home} />

React.Js 15 Interview Task and Solutions

Task 1: Create a Parent component that passes a list of items as props to a Child component. The Child component should render the list.

Solution:

Child.js

```
import React from "react";

const Child = ({ items }) => {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>{item}</li>
      ))}
    </ul>
  );
};

export default Child;
```

Explanation:

1. The Child component receives the items prop from the Parent component.
2. The items array is mapped to render <li> elements.
3. A unique key is added to each <li> for React to track updates efficiently.

Parent.js

```
import React from "react";
import Child from "./Child";

const Parent = () => {
  const itemList = ["Apple", "Banana", "Cherry"]; // Data to pass to the child
  return (
    <div>
      <h2>Items List:</h2>
      <Child items={itemList} /> {/* Pass itemList to Child as props */}
    </div>
  );
};

export default Parent;
```

Explanation:

1. The Parent component defines the itemList array.
2. This array is passed to the Child component via the items prop.

React.Js 15 Interview Task and Solutions

Task 2: Create a Child component that takes input from the user and passes the data to the Parent component.

Solutions:

Child.js

```
import React from "react";

const Child = ({ sendData }) => {
  return (
    <input
      type="text"
      onChange={(e) => sendData(e.target.value)} // Call sendData when input changes
      placeholder="Type something"
    />
  );
};

export default Child;
```

Explanation:

1. The Child component receives the sendData callback prop from the parent.
2. When the user types in the input box, the onChange handler calls sendData with the input value.

Parent.js

```
import React from "react";
import Child from "./Child";

const Parent = () => {
  const handleData = (data) => {
    console.log("Received from child:", data); // Log data from the child
  };

  return (
    <div>
      <h2>Data Sharing Example</h2>
      <Child sendData={handleData} /> {/* Pass handleData as a prop */}
    </div>
  );
};

export default Parent;
```

Explanation:

1. The Parent component defines handleData, which logs or processes the data received from the child.
2. The handleData function is passed to the Child component as sendData.

Task 3: Create two sibling components, InputBox and DisplayBox, that share data through their common parent component.

Solutions:

InputBox.js

```
import React from "react";

const InputBox = ({ updateData }) => {
  return (
    <input
      type="text"
      onChange={(e) => updateData(e.target.value)} // Call updateData on input change
      placeholder="Enter text"
    />
  );
};

export default InputBox;
```

Explanation:

1. The InputBox component receives the updateData prop from the parent.
2. It triggers updateData whenever the input changes.

DisplayBox.js

```
import React from "react";

const DisplayBox = ({ data }) => {
  return <p>Data: {data}</p>; // Display the shared data
};

export default DisplayBox;
```

Explanation:

1. The DisplayBox component receives the data prop from the parent.
2. It displays the value of data inside a <p> tag.

Parent.js

```
import React, { useState } from "react";
import InputBox from "./InputBox";
import DisplayBox from "./DisplayBox";
```

```
const Parent = () => {
  const [sharedData, setSharedData] = useState(""); // Shared state

  return (
    <div>
      <InputBox updateData={setSharedData} />{" "}
      {/* Pass setSharedData to InputBox */}
      <DisplayBox data={sharedData} /> {/* Pass sharedData to DisplayBox */}
    </div>
  );
};

export default Parent;
```

Explanation:

1. The Parent component uses the useState hook to manage sharedData.
2. The setSharedData function is passed to the InputBox to update the state.
3. The sharedData is passed to the DisplayBox for display.

Task 4: Create a Navbar component that displays different links based on whether a user is logged in.

Solutions:

Navbar.js

```
import React from "react";

const Navbar = ({ isLoggedIn }) => {
  return (
    <nav>
      <ul>
        {isLoggedIn ? (
          <>
            <li>
              <a href="/dashboard">Dashboard</a>
            </li>
            <li>
              <a href="/logout">Logout</a>
            </li>
          </>
        ) : (
          <>
            <li>
              <a href="/login">Login</a>
            </li>
            <li>
              <a href="/signup">Sign Up</a>
            </li>
```

```
        </>
      )}
     </ul>
   </nav>
 );
};

export default Navbar;
```

Explanation:

1. The isLoggedIn prop determines whether to show **dashboard/logout** links or **login/signup** links.
2. Conditional rendering is done using **fragments (<>...</>)** for grouping elements.

App.js

```
import React from "react";
import Navbar from "./Navbar";

const App = () => {
  return (
    <div>
      <h2>Conditional Rendering with Login State</h2>
      <Navbar isLoggedIn={true} /> {/* User is logged in */}
      <Navbar isLoggedIn={false} /> {/* User is logged out */}
    </div>
  );
};

export default App;
```

Task 5: Create a ToggleBox component that shows or hides content when a button is clicked.

Solutions:

ToggleBox.js

```
import React, { useState } from "react";

const ToggleBox = () => {
  const [isVisible, setIsVisible] = useState(true);

  const toggleVisibility = () => {
    setIsVisible((prev) => !prev);
  };

  return (
```

```
    <div>
      <button onClick={toggleVisibility}>
        {isVisible ? "Hide Content" : "Show Content"}
      </button>
      {isVisible && <p>This is some toggleable content.</p>}
    </div>
  );
};

export default ToggleBox;
```

Explanation:

1. useState is used to manage the isVisible state.
2. Clicking the button toggles isVisible between true and false.
3. Conditional rendering is achieved using the **logical AND (&&) operator**.

App.js

```
import React from "react";
import ToggleBox from "./ToggleBox";

const App = () => {
  return (
    <div>
      <h2>Show/Hide Content Example</h2>
      <ToggleBox />
    </div>
  );
};

export default App;
```

Task 6: Create a Counter component that increments and decrements a count value.

Solutions:

Counter.js

```
import React, { useState } from "react";

const Counter = () => {
  const [count, setCount] = useState(0); // Initialize state with 0

  const increment = () => {
    setCount((prevCount) => prevCount + 1); // Increment by 1
  };

  const decrement = () => {
    setCount((prevCount) => prevCount - 1); // Decrement by 1
```

```
  };

  return (
    <div>
      <h3>Count: {count}</h3>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
    </div>
  );
};

export default Counter;
```

Explanation:

1. useState(0) initializes the count state with 0.
2. increment and decrement functions update the state using the functional form of setCount to avoid stale state issues.
3. Buttons trigger these functions to modify and display the count

Task 7: Create a Form component to handle and display user input for their name.

Solutions:

Form.js

```
import React, { useState } from "react";

const Form = () => {
  const [name, setName] = useState(""); // Initialize state with an empty string

  const handleChange = (e) => {
    setName(e.target.value); // Update state with input value
  };

  return (
    <div>
      <input
        type="text"
        placeholder="Enter your name"
        value={name}
        onChange={handleChange}
      />
      <p>Your name is: {name}</p>
    </div>
  );
};

export default Form;
```

Explanation:

1. useState("") initializes the name state to an empty string.
2. The handleChange function updates the state with the value from the input field.
3. The input field's value is controlled by the name state

Task 8: Create a TodoList component to manage a list of tasks. Allow users to add tasks dynamically.

Solutions:

TodoList.js

```
import React, { useState } from "react";

const TodoList = () => {
  const [tasks, setTasks] = useState([]); // Initialize state with an empty array
  const [task, setTask] = useState(""); // State for current input

  const addTask = () => {
    if (task) {
      setTasks((prevTasks) => [...prevTasks, task]); // Add task to the list
      setTask(""); // Clear input field
    }
  };

  return (
    <div>
      <input
        type="text"
        value={task}
        placeholder="Enter a task"
        onChange={(e) => setTask(e.target.value)}
      />
      <button onClick={addTask}>Add Task</button>
      <ul>
        {tasks.map((t, index) => (
          <li key={index}>{t}</li> // Render each task
        ))}
      </ul>
    </div>
  );
};

export default TodoList;
```

Explanation:

1. useState([]) initializes tasks as an empty array.
2. setTasks updates the list by appending new tasks using the spread operator.

3. The input field's value is managed with the task state.

Task 9: Create a UserList component that fetches and displays user data from an API when the component mounts.

Solution:

UserList.js

```
import React, { useState, useEffect } from "react";

const UserList = () => {
  const [users, setUsers] = useState([]); // State to hold user data
  const [loading, setLoading] = useState(true); // State to track loading

  useEffect(() => {
    // Fetch data on component mount
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => response.json())
      .then((data) => {
        setUsers(data); // Set user data
        setLoading(false); // Stop loading
      })
      .catch((error) => {
        console.error("Error fetching data:", error);
        setLoading(false); // Stop loading in case of error
      });
  }, []); // Empty dependency array ensures this runs only once on mount

  return (
    <div>
      <h2>User List</h2>
      {loading ? (
        <p>Loading...</p>
      ) : (
        <ul>
          {users.map((user) => (
            <li key={user.id}>{user.name}</li>
          ))}
        </ul>
      )}
    </div>
  );
};

export default UserList;
```

Explanation:

1. useEffect runs the fetch logic once when the component mounts (empty dependency array []).
2. Data is fetched from an API, and the response is stored in the users state.
3. The loading state is used to display a loading message while the data is being fetched.

Task 10: Create a Timer component that updates a counter every second using useEffect.

Solution:

Timer.js

```
import React, { useState, useEffect } from "react";

const Timer = () => {
  const [seconds, setSeconds] = useState(0); // State for timer

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds((prev) => prev + 1); // Increment timer every second
    }, 1000);

    return () => clearInterval(interval); // Cleanup on component unmount
  }, []); // Empty dependency array ensures it runs once

  return (
    <div>
      <h2>Timer: {seconds} seconds</h2>
    </div>
  );
};

export default Timer;
```

Explanation:

1. useEffect sets up an interval when the component mounts.
2. The setInterval callback increments the seconds state every second.
3. The cleanup function (clearInterval) ensures the interval is cleared when the component unmounts.

Task 11: Create a ContactForm component that submits user details to an API using Axios.

Solution:

ContactForm.js

```
import React, { useState } from "react";
import axios from "axios";
```

```
const ContactForm = () => {
  const [name, setName] = useState(""); // State for name
  const [email, setEmail] = useState(""); // State for email
  const [message, setMessage] = useState(""); // State for message
  const [status, setStatus] = useState(""); // State for submission status

  const handleSubmit = (e) => {
    e.preventDefault(); // Prevent page refresh
    axios
      .post("https://jsonplaceholder.typicode.com/posts", {
        name,
        email,
        message,
      }) // Send POST request
      .then((response) => {
        setStatus("Form submitted successfully!"); // Success message
        setName(""); // Clear form fields
        setEmail("");
        setMessage("");
      })
      .catch((err) => {
        setStatus(`Error: ${err.message}`); // Error message
      });
  };

  return (
    <div>
      <h2>Contact Us</h2>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          required
        />
        <input
          type="email"
          placeholder="Email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />
        <textarea
          placeholder="Message"
          value={message}
          onChange={(e) => setMessage(e.target.value)}
```

```
        required
      ></textarea>
      <button type="submit">Submit</button>
    </form>
    <p>{status}</p>
  </div>
 );
};

export default ContactForm;
```

Task 12: Create a Dashboard with nested routes for "Profile" and "Settings."

Solution:

App.js

```
import React from "react";
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import Dashboard from "./Dashboard";
import Home from "./Home";

const App = () => {
 return (
  <Router>
   <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/dashboard/*" element={<Dashboard />} />
   </Routes>
  </Router>
 );
};

export default App;
```

Dashboard.js

```
import React from "react";
import { Routes, Route, Link } from "react-router-dom";
import Profile from "./Profile";
import Settings from "./Settings";

const Dashboard = () => {
 return (
  <div>
   <h2>Dashboard</h2>
   <nav>
    <ul>
```

```
        <li>
          <Link to="profile">Profile</Link>
        </li>
        <li>
          <Link to="settings">Settings</Link>
        </li>
      </ul>
    </nav>
    <Routes>
      <Route path="profile" element={<Profile />} />
      <Route path="settings" element={<Settings />} />
    </Routes>
  </div>
 );
};

export default Dashboard;
```

Profile.js

```
const Profile = () => <h3>Profile Page</h3>;
export default Profile;
```

Settings.js

```
const Settings = () => <h3>Settings Page</h3>;
export default Settings;
```

Explanation:

1. **Problem Solved:**
   - Organizes a parent-child relationship for nested routes.
2. **Solution Highlights:**
   - The Dashboard component contains routes for its child components (Profile, Settings).
   - Relative paths (profile and settings) simplify navigation.
3. **Why Nested Routes?**
   - Models hierarchical navigation.

Task 13: Create a "404 Not Found" page for undefined routes.

Solution:

App.js

```
import React from "react";
```

```
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import Home from "./Home";
import NotFound from "./NotFound";

const App = () => {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </Router>
  );
};

export default App;
```

NotFound.js

```
const NotFound = () => <h2>404 - Page Not Found</h2>;
export default NotFound;
```

Explanation:

1. **Problem Solved:**
   - Handles invalid or undefined routes.
2. **Solution Highlights:**
   - The * route acts as a catch-all for undefined paths.
3. **Why Handle 404?**
   - Improves user experience by informing them of navigation errors.

Task 14: Write code for React component for a Credit Card Number Validator, commonly used to verify the validity of a credit card number.

Solution:

CreditCardValidator.js

```
import React, { useState } from "react";

const CreditCardValidator = () => {
  const [cardNumber, setCardNumber] = useState("");
  const [isValid, setIsValid] = useState(null);

  // Luhn Algorithm to validate credit card number
  const validateCardNumber = (number) => {
    const digits = number.split("").reverse().map(Number);
    const checksum = digits.reduce((sum, digit, index) => {
      if (index % 2 === 1) {
        let doubled = digit * 2;
```

```
      if (doubled > 9) doubled -= 9;
      return sum + doubled;
    }
    return sum + digit;
  }, 0);

  return checksum % 10 === 0;
};

const handleSubmit = (e) => {
  e.preventDefault();
  if (cardNumber.length < 13 || cardNumber.length > 19) {
    setIsValid(false); // Basic length check
  } else {
    setIsValid(validateCardNumber(cardNumber));
  }
};

return (
  <div>
    <h2>Credit Card Validator</h2>
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Enter Credit Card Number"
        value={cardNumber}
        onChange={(e) => setCardNumber(e.target.value.replace(/\D/g, ""))} // Allow only
digits
      />
      <button type="submit">Validate</button>
    </form>
    {isValid === null ? null : isValid ? (
      <p style={{ color: "green" }}>Valid Credit Card Number</p>
    ) : (
      <p style={{ color: "red" }}>Invalid Credit Card Number</p>
    )}
  </div>
);
};

export default CreditCardValidator;
```

How It Works:

1. **Input Handling:**
   - The input field only allows numeric values using e.target.value.replace(/\D/g, "").
2. **Validation Logic:**
   - The validateCardNumber function implements the **Luhn Algorithm**:

- Reverse the card number.
- Double every second digit.
- Subtract 9 from any result greater than 9.
- Sum all the digits.
- If the total modulo 10 is 0, the card is valid.

3. **Submission Handling:**
   - The handleSubmit function checks the length of the card number (13-19 digits are standard for most cards).
   - If valid, the Luhn Algorithm is applied for final validation.
4. **Feedback:**
   - Displays whether the card number is valid or invalid.

Usage Example:

- Input: 4111111111111111 (a common test number for Visa cards)
- Output: **Valid Credit Card Number**

Task 15: Write a React component to implement a stopwatch with the following functionalities:

1. **Start the timer.**
2. **Pause the timer.**
3. **Reset the timer to zero.**

Solution:

Stopwatch.js

```
import React, { useState, useEffect } from "react";

const Stopwatch = () => {
  const [time, setTime] = useState(0); // State to track time in seconds
  const [isRunning, setIsRunning] = useState(false); // State to track if stopwatch is running

  useEffect(() => {
    let interval;
    if (isRunning) {
      interval = setInterval(() => {
        setTime((prevTime) => prevTime + 1);
      }, 1000);
    } else if (!isRunning && time !== 0) {
      clearInterval(interval);
    }
    return () => clearInterval(interval); // Cleanup interval on unmount
  }, [isRunning]);

  const handleStart = () => setIsRunning(true); // Start the stopwatch
  const handlePause = () => setIsRunning(false); // Pause the stopwatch
  const handleReset = () => {
```

```
  setIsRunning(false);
  setTime(0); // Reset time to zero
 };

 const formatTime = (seconds) => {
  const minutes = Math.floor(seconds / 60);
  const secs = seconds % 60;
  return `${String(minutes).padStart(2, "0")}:${String(secs).padStart(
    2,
    "0"
  )}`;
 };

 return (
  <div>
    <h2>Stopwatch</h2>
    <h3>{formatTime(time)}</h3> {/* Display time in MM:SS format */}
    <button onClick={handleStart} disabled={isRunning}>
      Start
    </button>
    <button onClick={handlePause} disabled={!isRunning}>
      Pause
    </button>
    <button onClick={handleReset}>Reset</button>
  </div>
 );
};

export default Stopwatch;
```

Explanation:

Problem Breakdown:

1. **What to Implement?**
   - A stopwatch that can:
     - Start counting time in seconds.
     - Pause the counting.
     - Reset the time to zero.
2. **Expected Behavior:**
   - Clicking "Start" begins the timer.
   - Clicking "Pause" pauses the timer.
   - Clicking "Reset" stops the timer and resets it to zero.

Solution Details:

1. **State Management:**
   - time: Tracks elapsed time in seconds.

    - isRunning: Indicates whether the stopwatch is running.
2. **Effect Hook (useEffect):**
    - Sets up an interval to update the time state every second if isRunning is true.
    - Cleans up the interval when isRunning is false or the component unmounts.
3. **Handlers:**
    - handleStart: Sets isRunning to true to start the stopwatch.
    - handlePause: Sets isRunning to false to pause the stopwatch.
    - handleReset: Resets both time and isRunning.
4. **Time Formatting:**
    - The formatTime function converts seconds into a MM:SS format for display.

How It Works:

1. **Start:**
    - Clicking "Start" begins the timer by enabling isRunning and initiating an interval that increments time.
2. **Pause:**
    - Clicking "Pause" clears the interval, stopping the time increment.
3. **Reset:**
    - Clicking "Reset" stops the timer and resets the time to zero.
4. **Display:**
    - The time is displayed in MM:SS format, ensuring a clean UI.